Resumen Maestro: Lectura 7 — BinaryHeap, Colas de Prioridad y Heapify

> **Curso:** CC232 - Algoritmos y Estructuras de Datos (UNI)
> **Cátedra:** Prof. César Lara Avila
> **Semana:** 07 — Montículos Binarios y Mantenimiento del Mínimo
> **Archivo Original:** Lectura7.md

---

1. La Cadena Retrospectiva y Motivación de Ingeniería

```
problema -> mantener mínimo -> ADT PriorityQueue -> arreglo implícito -> bubbleUp/trickleDown -> heapi
```

En las semanas previas diseñamos estructuras para búsquedas dinámicas de cualquier clave: el BST simple ($O(h)$) y el AVL ($O(\log n)$). Ambas estructuras imponen un **invariante global de orden** (todo subárbol izquierdo menor, todo subárbol derecho mayor).

En la Semana 7 cambia por completo el objetivo de diseño: **¿qué ocurre si en nuestro sistema no necesitamos buscar claves arbitrarias, sino procesar tareas según su urgencia y consultar inmediatamente el elemento mínimo?**

- En un arreglo desordenado, insertar es $O(1)$, pero encontrar o retirar el mínimo exige un escaneo lineal completo de $O(n)$.
- En un arreglo totalmente ordenado, el mínimo está en la primera celda en $O(1)$, pero insertar un nuevo elemento exige desplazar celdas en $O(n)$.

La **Cola de Prioridad (Priority Queue)** resuelve este compromiso manteniendo **únicamente el orden estrictamente necesario** para que el elemento extremo (mínimo) esté accesible de inmediato. El **BinaryHeap (Montículo Binario)** implementa este ADT eliminando el gasto de punteros y de balanceo dinámico, codificando un árbol binario completo dentro de un arreglo contiguo en memoria.

---

2. La Quíntupla Estructural Canónica

1. **ADT:** Priority Queue Mínima (`boolean add(int x)` , `Integer peek()` , `int remove()` , `int size()`).
2. **Representación:** Dualidad árbol-arreglo. Se almacena en memoria contigua mediante `Integer[] a` y un contador de elementos lógicos `int n` .
   - **Mapeo implícito por niveles (sin punteros `left` , `right` ni `parent`):**

$$\text{left}(i) = 2i + 1 \qquad \text{right}(i) = 2i + 2 \qquad \text{parent}(i) = \lfloor (i-1)/2 \rfloor$$

- **Regla Sagrada de Existencia:** Un índice calculado aritméticamente **solo existe si está dentro del prefijo lógico**:

$$\text{hijo_izquierdo}(i) \text{ existe} \iff 2i + 1 < n$$

3. **Los Dos Invariantes Obligatorios Simultáneos:**
   - *Invariante de Forma:* **Árbol Binario Completo**. Todos los niveles están completamente llenos, excepto posiblemente el último, que se llena estrictamente de izquierda a derecha sin dejar huecos. Corresponde exactamente al prefijo contiguo `a[0..n-1]`.
     - *Garantía geométrica:* La altura está matemáticamente sellada en $h = \lfloor \log_2 n \rfloor = O(\log n)$ independientemente de los valores que se inserten.
   - *Invariante de Prioridad (Min-Heap):* Para todo índice lógico $i > 0$:

$$a[\text{parent}(i)] \leq a[i] \quad (\text{Cada padre es menor o igual que sus hijos})$$

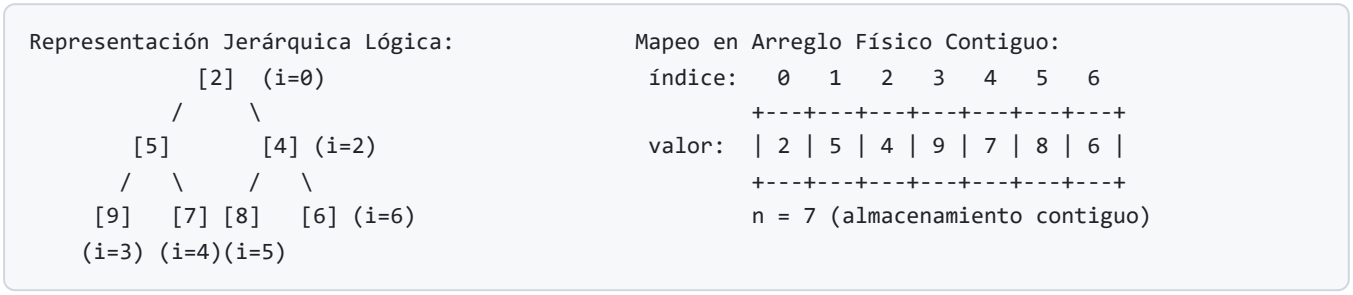
- *Consecuencia Inmediata:* Siguiendo la cadena de ancestros hasta la cima, la raíz contiene obligatoriamente el elemento mínimo global:

$$\min(a[0..n-1]) = a[0] \implies \mathbf{peek}() \to \mathbf{O(1)}$$

4. **Mecanismos de Reparación:**
   - `bubbleUp(i)`: Reparar hacia arriba tras insertar al final ($O(\log n)$).
   - `trickleDown(i)`: Reparar hacia abajo tras extraer la raíz ($O(\log n)$).
   - `heapify()`: Construcción en lote de abajo hacia arriba en tiempo lineal ($O(n)$).
5. **Complejidad Asintótica:** `peek()` es $O(1)$; `add()` y `remove()` son $O(\log n)$; `heapify()` es $O(n)$. Buscar una clave arbitraria es $O(n)$ (el heap no es un BST).

---

3. La Dualidad Árbol Binario Completo ↔ Arreglo



No hay referencias `next`, `prev`, `left`, `right` ni `parent`. El hardware aprovecha la **localidad de referencia espacial (los datos vecinos en memoria física se cargan juntos en la caché L1/L2 del procesador)**, logrando una velocidad de ejecución muy superior a la de los árboles con nodos dispersos en el heap.

Métodos Auxiliares de Navegación, Memoria y Consulta:

```
// Navegación aritmética O(1)
private int left(int i)   { return 2 * i + 1; }
private int right(int i)  { return 2 * i + 2; }
private int parent(int i) { return (i - 1) / 2; }

// Intercambio de celdas en memoria O(1)
private void swap(int i, int j) {
    int temp = a[i];
    a[i] = a[j];
    a[j] = temp;
}

// Redimensionamiento geométrico amortizado O(1)
private void resize() {
    a = Arrays.copyOf(a, Math.max(1, 2 * n));
}

// Consultas directas O(1)
public int size() {
    return n;
}

public Integer peek() {
    return (n == 0) ? null : a[0]; // Mínimo garantizado en la raíz
}
```

---

4. Las Operaciones de Mutación: "Forma Primero, Prioridad Después"

Toda modificación en un `BinaryHeap` se realiza en dos tiempos estrictos: primero se preserva la forma completa en $O(1)$, y luego se repara el orden en $O(\log n)$.

A. Inserción (`add(int x)`): Forma al final + `bubbleUp`

1. **Preservar Forma:** Escribir el nuevo elemento en la primera posición vacía: `a[n] = x; n++;` (con redimensionamiento geométrico si `n == a.length`).
2. **Reparar Prioridad (`bubbleUp(n - 1)`):** El nuevo elemento puede ser menor que su padre, violando la regla $a[\text{parent}] \leq a[i]$. Se compara y asciende mediante intercambios:

```
public boolean add(int x) {
    if (n + 1 > a.length) {
        resize(); // Duplica capacidad si se llena el arreglo físico
    }
    a[n++] = x;       // 1. Forma: insertar como nueva hoja al final
    bubbleUp(n - 1);  // 2. Prioridad: reparar hacia arriba
    return true;
}

private void bubbleUp(int i) {
    while (i > 0 && a[i] < a[parent(i)]) {
        int p = parent(i);
        swap(i, p);
        i = p; // Continuar subiendo
    }
}
```

Costo: Sube como máximo a lo largo de un camino vertical: $O(\log n)$.

B. Extracción del Mínimo (`remove()`): Sustitución + `trickleDown`

1. Guardar el valor a retornar: `int min = a[0];`.
2. **Preservar Forma:** Si quitamos la raíz directamente queda un hueco. Para mantener la forma compacta sin mover todos los elementos, **se traslada el último elemento lógico a la raíz**: `a[0] = a[--n]; a[n] = null;`.
3. **Reparar Prioridad (`trickleDown(0)`):** El valor recién colocado en la raíz suele ser grande y viola el invariante con sus hijos. Debe descender intercambiándose **estrictamente con el menor de sus hijos válidos**:

```
public int remove() {
    if (n == 0) {
        throw new NoSuchElementException("el heap está vacío");
    }
    int min = a[0];
    a[0] = a[--n]; // Mover el último elemento a la raíz
    a[n] = null;   // Limpiar referencia para el recolector de basura

    if (n > 0) {
        trickleDown(0); // Reparar prioridad hacia abajo
    }

    if (a.length >= 3 * n) {
        resize(); // Reducir capacidad si está subutilizado
    }
    return min;
}

private void trickleDown(int i) {
    while (true) {
        int smallest = i;
        int l = left(i);
        int r = right(i);

        // Validar existencia y comparar con el hijo izquierdo
        if (l < n && a[l] < a[smallest]) {
            smallest = l;
        }

        // Validar existencia y comparar con el mejor candidato actual
        if (r < n && a[r] < a[smallest]) {
            smallest = r;
        }

        // Si el nodo actual ya es menor o igual que sus hijos, terminamos
        if (smallest == i) {
            return;
        }

        swap(i, smallest);
        i = smallest; // Continuar descendiendo
    }
}
```

CUIDADO / TRAMPA FRECUENTE

¿Por qué intercambiar obligatoriamente con el MENOR de los dos hijos? Supón que la raíz tiene valor 11 y sus hijos son 5 y 4. Si intercambias con el 5, la raíz queda con 5 y su hijo derecho sigue siendo 4, produciendo inmediatamente una violación ($5 \leq 4$ es falso). Al elegir el menor (4), garantizas que el nuevo padre sea menor que ambos subárboles.

5. Floyd's `heapify()`: Construcción en Lote en $O(n)$

Si recibes un arreglo desordenado de n elementos y deseas convertirlo en un min-heap:

- **Estrategia Ingenua:** Empezar vacío e insertar uno por uno con `add(x)` $\rightarrow n \times O(\log n) = \mathbf{O(n \log n)}$.
- **Estrategia Óptima de Floyd (`heapify`):** Observar que el arreglo ya tiene forma de árbol completo. Solo falta reparar las prioridades.

El Secreto: Comenzar desde el Último Nodo Interno

Las hojas de un árbol son, por definición, heaps trivialmente válidos (no tienen hijos con quienes violar el invariante). No necesitan reparación. En un arreglo de tamaño n, las hojas ocupan las posiciones desde $\lfloor n/2 \rfloor$ hasta $n-1$. Por lo tanto, el **último nodo con al menos un hijo** se encuentra en:

$$\text{último nodo interno} = \lfloor n/2 \rfloor - 1$$

El algoritmo procesa los nodos internos de **abajo hacia arriba (orden descendente de índices)**:

```
// Constructor en lote: construye el heap en O(n) tiempo total
public BinaryHeap(Integer[] values) {
    a = Arrays.copyOf(values, Math.max(1, values.length));
    n = values.length;
    heapify(); // Transforma el arreglo crudo en un min-heap en O(n)
}

private void heapify() {
    for (int i = n / 2 - 1; i >= 0; i--) {
        trickleDown(i);
    }
}
```

Distinción Técnica: Profundidad (hacia abajo) frente a Altura (hacia arriba)

- **Profundidad (Depth):** Distancia desde la raíz. La raíz está en profundidad 0 y las hojas en profundidad $\approx \log n$.
- **Altura de un Nodo en Heapify (h):** Distancia máxima que un nodo puede descender hacia las hojas. Las hojas están a **altura** $h = 0$ (no bajan nada). Sus padres están a **altura** $h = 1$ (bajan a lo más 1 nivel con `trickleDown`), y la raíz está a **altura** $h = \lfloor \log_2 n \rfloor$.
- Esta distinción es la que explica el milagro de Floyd: **el trabajo de `trickleDown` no es proporcional a la profundidad del nodo, sino a su altura** h.

¿Por qué `heapify()` es $O(n)$ y NO $O(n \log n)$? (Demostración de Examen)

Aunque una llamada individual a `trickleDown` puede costar $O(\log n)$, **la inmensa mayoría de los nodos están en los niveles inferiores y tienen caminos de descenso casi nulos**:

```
Nivel (desde base)   Nodos aproximados   Descenso máximo por nodo
-----------------------------------------------------------------
h = 0 (hojas)        n / 2               0 niveles (ni se tocan)
h = 1                n / 4               1 nivel
h = 2                n / 8               2 niveles
...                  ...                 ...
h = log n (raíz)     1 nodo              log n niveles
```

El trabajo total es la suma ponderada:

$$T(n) = \sum_{h=0}^{\lfloor \log n \rfloor} \frac{n}{2^{h+1}} \cdot h = \frac{n}{2} \sum_{h=0}^{\infty} \frac{h}{2^h}$$

La serie matemática $\sum_{h=0}^{\infty} \frac{h}{2^h}$ converge exactamente a la constante 2. Sustituyendo:

$$T(n) = \frac{n}{2} \cdot 2 = \mathbf{O(n)}$$

Conclusión de examen: `heapify` es lineal porque **hay muchísimos nodos baratos que bajan 0 o 1 nivel, y un solo nodo caro (la raíz) que puede bajar $\log n$ niveles**.

---

6. Contrastes Conceptuales Clave

A. BinaryHeap frente a Árbol Binario de Búsqueda (BST)

- **Invariante:** El heap tiene una regla **local** ($a[\text{padre}] \leq a[\text{hijo}]$); el BST tiene una regla **global** ($\text{todo subárbol izq} < \text{nodo} < \text{todo subárbol der}$).
- **Búsqueda Arbitraria:** En BST cuesta $O(\log n)$ porque cada comparación descarta la mitad del árbol. En un min-heap, saber que $x > \text{raíz}$ no dice nada sobre si ir a la izquierda o derecha: **cuesta $O(n)$**.
- **Duplicados:** El heap acepta valores duplicados de forma natural con $\leq$; los BST didácticos requieren políticas explícitas.

B. PriorityQueue frente a Queue (Semana 3)

- `Queue` es estrictamente FIFO (primero en llegar, primero en salir).
- `PriorityQueue` ignora el orden de llegada y despacha estrictamente por valor de prioridad.

---

7. Trampas Fijas de Examen (Catálogo de Errores UNI)

1. **Creer que el arreglo del heap está totalmente ordenado:** En un min-heap válido `[2, 5, 4, 9, 7, 8, 6]`, el elemento 5 precede al 4 en memoria. El heap solo garantiza orden entre ancestro y descendiente, no entre hermanos o primos.
2. **Acceder a hijos sin verificar `< n`:** Calcular `int l = 2 * i + 1;` y leer directamente `a[l]`. Si $l \geq n$, estás leyendo basura fuera del tamaño lógico o provocando un `ArrayIndexOutOfBoundsException`.
3. **Intercambiar con el hijo incorrecto en `trickleDown`:** Elegir el hijo derecho solo porque es menor que el padre, sin comprobar si el hijo izquierdo era todavía menor.
4. **Afirmar que construir un heap siempre cuesta $O(n \log n)$:** Con inserciones individuales es $O(n \log n)$, pero con Floyd's `heapify()` desde $\lfloor n/2 \rfloor - 1$ es estrictamente $O(n)$.

5. **Creer que `remove()` borra la celda del final sin moverla:** Si borras la raíz directamente, rompes la contigüidad. Siempre se copia `a[n-1]` a `a[0]` y se decrementa `n`.

8. Tabla Resumen de Complejidad Asintótica

Operación	Complejidad	Justificación Física y Algorítmica
`size()`	$O(1)$	Retorna la variable primitiva interna `n`.
`peek()`	$O(1)$	Lectura directa de la celda raíz `a[0]`.
`add(x)`	$O(\log n)$	Escritura en `a[n]` ($O(1)$ amortizado) + subida `bubbleUp` acotada por h.
`remove()`	$O(\log n)$	Reemplazo con `a[n-1]` + descenso `trickleDown` acotado por h.
`heapify()`	$O(n)$	Suma de alturas de abajo hacia arriba acotada por sumatoria geométrica.
Buscar clave x	$O(n)$	Requiere escanear el arreglo; no hay criterio de descarte direccional.

9. Comprueba tu Comprensión (Pausa de Trazado)

1. **Traza de Heapify:** Dado el arreglo crudo `a = [9, 4, 7, 1, 3, 6, 2]` con $n = 7$:
   - ¿Cuál es el índice del último nodo interno ($\lfloor n/2 \rfloor - 1$)?
   - Trazar los intercambios que realiza `heapify()` para $i = 2$, luego $i = 1$, y finalmente $i = 0$.
2. **Cálculo de Índices:** Si un heap tiene $n = 12$ elementos:
   - ¿Cuáles son los índices de los hijos del nodo en la posición $i = 5$? ¿Existen ambos lógicamente?
   - ¿Quién es el padre del nodo en la posición $i = 8$?
3. **Extracción:** Sobre el min-heap `[2, 5, 4, 9, 7, 8, 6]`, traza paso a paso la ejecución de `remove()`. ¿Qué valor sube a la raíz temporalmente y qué intercambios ocurren?

> *Resuelve estos tres ejercicios a mano en tu cuaderno antes de entrar a las preguntas de examen.*